Testing ON DELETE SET NULL

Lecture Notes · Game DB Junior

In this lesson

- How to set up **ON DELETE SET NULL**
- Why an INSERT can fail after a user is deleted
- Testing SET NULL and when to use it

1. Setting up SET NULL

Again, we **drop** the photos table and create it again. This time, after the foreign key column, we write **ON DELETE SET NULL**:

```
DROP TABLE photos;

CREATE TABLE photos (
    id SERIAL PRIMARY KEY,
    url VARCHAR(200),
    user_id INTEGER REFERENCES users(id) ON DELETE SET NULL
);
```

2. Watch out: user 1 no longer exists

- Our old INSERT statement adds three photos for the user with **id 1**.
- But we **deleted** user 1 in the last lesson. That user does not exist anymore.
- So that INSERT would give an **error**: there is no user to relate these photos to.
- For this test, we **remove those three lines** and insert only the other photos.

3. Testing SET NULL

This time we delete the user with **id 4**. Because of ON DELETE SET NULL, all the photos of this user should get **NULL** in the user_id column.

```
DELETE FROM users
WHERE id = 4;
```

- The delete is successful.
- With **SELECT * FROM photos;** we see **two photos** with a user_id of NULL.

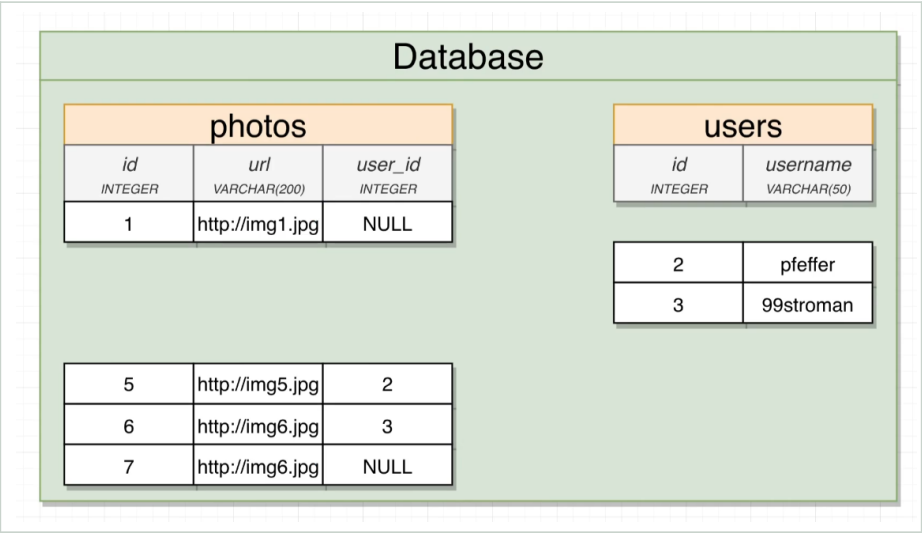


Figure 1 — After deleting user 4, photos 1 and 7 have user_id NULL

4. When to use SET NULL

SET NULL is useful when we delete a user but still want to **keep what they created**.

- Example: in an app like Instagram, we might need to delete or ban a problematic user.
- But the posts they made might be very meaningful, so we want to keep them.
- So we keep the related records, even though the record they depend on is deleted.

5. Running the code (my practice)

Step 1: DROP TABLE photos. Now only cities and users are left.

```
-- Delete (drop) the whole photos table and all the data in it
-- We do this so we can create it again with a different ON DELETE option
DROP TABLE photos;
```

Figure 2 — DROP TABLE photos



Figure 3 — Tables (2)

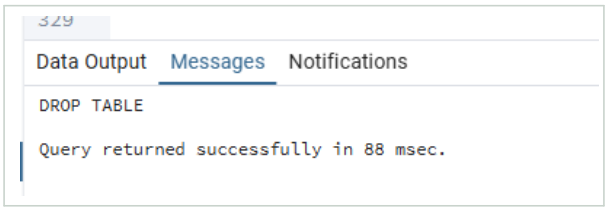


Figure 4 — DROP TABLE ran successfully

Step 2: my old INSERT has photos for user 1. User 1 was deleted in the last lesson, so these lines would cause an error.

```sql
-- Add 7 photos to the photos table
INSERT INTO photos (url, user_id)
VALUES
    ('http:/one.jpg', 4),   -- id 1, user 4
    ('http:/two.jpg', 1),   -- id 2, user 1
    ('http:/25.jpg', 1),    -- id 3, user 1
    ('http:/36.jpg', 1),    -- id 4, user 1 (user 1 has 3 photos)
    ('http:/754.jpg', 2),   -- id 5, user 2
    ('http:/35.jpg', 3),    -- id 6, user 3
    ('http:/256.jpg', 4);   -- id 7, user 4 (user 4 has 2 photos)
```

Figure 5 — The old INSERT (has 3 photos for user 1)

Step 3: create the table with ON DELETE SET NULL. I removed the 3 lines for user 1 and inserted 4 photos.

```sql
-- Create the photos table again (after dropping it)
CREATE TABLE photos (
    id SERIAL PRIMARY KEY,
    url VARCHAR(200),
    -- ON DELETE SET NULL: when a user is deleted,
    -- the photos of that user stay, but their user_id becomes NULL
    user_id INTEGER REFERENCES users(id) ON DELETE SET NULL
);

-- Add 7 photos to the photos table
INSERT INTO photos (url, user_id)
VALUES
    ('http:/one.jpg', 4),   -- id 1, user 4
    ('http:/754.jpg', 2),   -- id 5, user 2
    ('http:/35.jpg', 3),    -- id 6, user 3
    ('http:/256.jpg', 4);   -- id 7, user 4 (user 4 has 2 photos)
```

Figure 6 — CREATE TABLE with ON DELETE SET NULL, then INSERT

The Messages tab shows **INSERT 0 4**. Now there are 3 tables again.

```
INSERT 0 4

Query returned successfully in 73 msec.
```

Figure 7 — INSERT 0 4

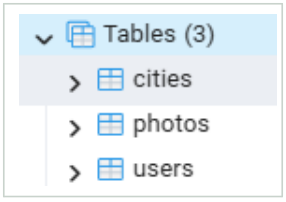


Figure 8 — Tables (3)

Step 4: delete the user with id 4. The Messages tab shows **DELETE 1**.

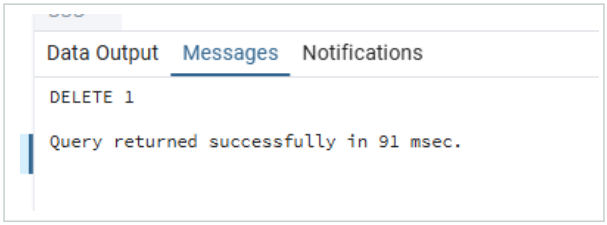


Figure 9 — DELETE 1

Step 5: check users and photos. Only users 2 and 3 are left. The two photos of user 4 (one.jpg and 256.jpg) are still there, but their user_id is now **[null]**.

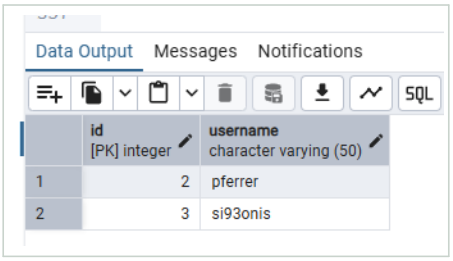


Figure 10 — users after the delete

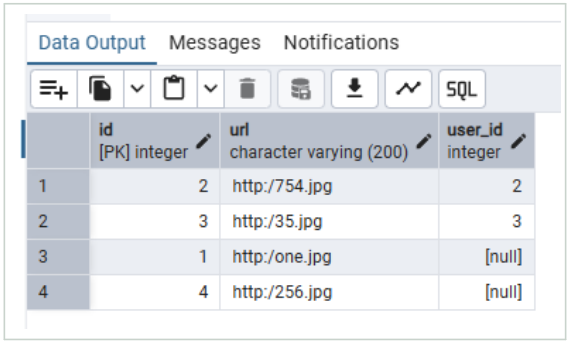


Figure 11 — photos: user_id is [null]

Note: the new table started counting ids from 1 again, so my 4 photos got ids 1 to 4 (not 1, 5, 6, 7 like in my comments). Also, in my users table id 4 was 99stroman, so that is the user I deleted.

6. Summary

Idea	Remember
ON DELETE SET NULL	Deleting a user keeps their photos, but sets user_id to NULL
Where it goes	Right after the foreign key column: REFERENCES users(id) ON DELETE SET NULL
Deleted users	You cannot insert rows that point at a user that was deleted
Good use case	Delete or ban a user, but keep their meaningful posts

Next lesson: default values and ON DELETE SET DEFAULT.

Sources: Lecture transcript and lecture image (Figure 1); my own practice screenshots (Figures 2–11)